# מודול 4 · מחזור חיי המוצר בעזרת AI

## גרסת שעתיים · מדריך למנחה

זוהי גרסת השעתיים של מודול 4. היא מספרת את אותו סיפור, עם אותו קוד ואותם שקפים כמו המודול בן ארבע השעות, ושומרת על ההסברים שלו. במקום משימות בזוגות, **את כל השלבים מבצעים בשידור חי עם העוזר, בזמן שהכיתה צופה ומנבאת**.

**בנוי למסך מוקרן אחד.** כל השיעור מתנהל בחלון VS Code אחד, על סביבת עבודה כיתתית אחת שפקודה אחת בונה לפני השיעור. כל שלב ממוספר ב- `class/prompts.md` : מעתיקים, מדביקים, מריצים. במדריך הזה, **פרומפט N** מסמן את הרגע להעתיק את רשומה N מתוך `prompts.md` – פרומפט לצ'אט או פקודה בטרמינל; מה שבא לפניו באותו שלב נאמר או נשאל קודם. המעבר לתחנה הבאה בסיפור הוא פקודה אחת, `python class/go.py STAGE` . על המסך עוברים בין המצגת ל-VS Code, ובבלוקים 1, 3 ו-4 גם לכרטיסיית דפדפן. בשיעור עצמו קוראים מהתמליל בעברית שבטלפון; המדריך הזה נועד להכנה.

**הסיפור בשלושה משפטים.**

1. פיצ'ר קטן – תאריכי יעד אופציונליים ותצוגת "Overdue" (באיחור) – מחייב שתי החלטות: איזה יום נחשב "היום" (היום לפי UTC), ומה המשמעות של ניקוי תאריך.
2. בדיקה שנכתבת מתוך ההחלטות האלה תופסת באג בניקוי התאריך, שהבדיקות הישנות פספסו.
3. אחרי השחרור, "סידור" קוד שעשה AI שובר בשקט את "היום" בשרת שמוגדר לשעון מקומי. כל הבדיקות עוברות, כי הבדיקות החליפו את השעון; בדיקה שמחליפה רק את השעון תופסת את הבאג.

**איך עובד כל בלוק.** מריצים את השלב בשידור חי עם העוזר, והכיתה מנבאת את התוצאה מראש. אחר כך מצביעים על מה שהתוצאה מראה, בעזרת "מה להדגיש". לכל קריאה חיה יש חלופה: פקודת `go` , שקף, או משהו שאומרים. אם תשובה לוקחת יותר מכ-60 שניות, עוברים לחלופה ואומרים זאת.

**דרך עבודה אחת עם העוזר.** כל פרומפט נשלח ל-Copilot Chat במצב Agent. קובץ הכללים של סביבת העבודה ( `AGENTS.md` , וגם ב- `.github/copilot-instructions.md` ) מורה לעוזר להציע כל שינוי בקוד או בבדיקות ולחכות לאישור. לכן כל שלב עריכה הוא שני פרומפטים: הבקשה, ואחריה "go ahead" קצר. זה גם הסדר של שקף 9: תכנון, אישור, עריכה, סקירה, בדיקה. פרומפטים שרק קוראים מסתיימים ב-"Do not edit files." שלושה הרגלים שומרים על הצ'אט נקי, ו- `prompts.md` מזכיר כל אחד מהם: לפתוח **צ'אט חדש** היכן ששלב מציין זאת, כדי שתשובות קודמות לא ייגררו הלאה; ללחוץ **Keep** אחרי עריכה שאושרה, לפני שממשיכים; ולהשאיר את השלב הנוכחי בראש `prompts.md` , כי שלבים מאוחרים יותר חושפים תשובות.

**מה התלמידים צריכים:** כלום. אף אחד לא מתקין או פורס שום דבר בשיעור.

**אחרי השיעור,** משתפים את ה-zip של התלמידים ( `student/` בחבילה) יחד עם ארבעת הקבצים שהמדריך שלו מצפה להם במהלך השיעור: `handouts/feature_contract.md` , `handouts/2_task_c_start.zip` , `handouts/task_e_thread_summary.md` ו- `handouts/task_f_release_0_4_1.zip` . עם אלה כל אחד יכול לבצע את שש המשימות בבית. שיעורי הבית משתמשים בראיות התמיכה המלאות יותר של גרסת ארבע השעות; התשובות נמצאות בסוף המדריך.

## במה משתמשים

| פריט | משמש ל- |
|---|---|
| `module4_slides_short.pptx` | המצגת: אותם 20 שקפים, במצב הצגה רגיל (אין צורך בתצוגת מציג) |
| `setup_class.py` | בונה את סביבת העבודה הכיתתית לפני השיעור, בפקודה אחת |
| `class/` | מועתקת לסביבת העבודה, במקום ש-Git ו-Copilot מתעלמים ממנו: `prompts.md` (כל השלבים), `show/` (שני קבצים שמציגים) ושני כלי הכיתה, `go.py` ו-`mutant.py`. הקובץ `class/inputs/support_thread.md` הוא המקור של שרשור התמיכה; התחנות של סביבת העבודה מביאות אותו, יחד עם הבקשה וה-backlog, בתיקיית `inputs/` גלויה |
| התמליל בעברית, בטלפון | מה לומר, שלב אחר שלב |
| המדריך הזה | הכנה: התוכנית, התוצאות הצפויות, מה להדגיש והחלופות |

## במבט אחד (שעתיים, בלי הפסקה)

| זמן | בלוק | שקפים | שלבים ב- `prompts.md` |
|---|---|---|---|
| 0:00–0:07 | 1 · פתיחה | 1–3 | 1 |
| 0:07–0:19 | 2 · דרישות | 4–5 | 2–3 |
| 0:19–0:31 | 3 · תכן | 6–8 | 4–5 |
| 0:31–0:57 | 4 · בנייה ובדיקות | 9–11 | 6–17 |
| 0:57–1:07 | 5 · שחרור | 12–13 | 18–20 |
| 1:07–1:19 | 6 · תחזוקה: מיון התקלה | 14–15 | 21–22 |
| 1:19–1:31 | 7 · סגירת המעגל | 16 | 23–30 |
| 1:31–1:39 | 8 · תיקון ומסירות | 17 | 31–32 |
| 1:39–1:49 | 9 · סיכום | 18–20 | — |
| 1:49–2:00 | עתודה לתשובות איטיות ולשאלות | | |

אין הפסקה, ולכן שומרים על קצב קבוע. אם מפגרים, מקצרים את החלקים שמסומנים (אפשר לקצר) – אף פעם לא את ההסברים.

## לפני השיעור (כ-10 דקות)

1. **הסביבה שלך.** מתיקיית חבילת השעתיים (זו שבה נמצאים `setup_class.py` , `class/` ו- `code/` ):

```
python -m venv .venv               # macOS: python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1 (PowerShell) or
.venv\Scripts\Activate.bat (cmd)
python -m pip install -r code/3_feature_0.4.0/requirements.lock
python code/tools/preflight.py
```

כל שורה של ה-preflight צריכה להציג `OK` ; שורת ה-Node עשויה להציג `SKIP` .

2. **סביבת העבודה הכיתתית.** כשהסביבה עדיין פעילה:

```
python setup_class.py ../module4_class
```

ממקמים אותה מחוץ לחבילה, וגם מחוץ למאגר ה-Git שלך אם החומרים נמצאים במאגר כזה. (בחבילת המנחה המלאה, שבה התיקייה הזו היא `short_2h/` , מריצים `python short_2h/setup_class.py ../module4_class` מתיקיית החבילה.) הפקודה מדפיסה את הצעדים הבאים. הרצה חוזרת בונה את סביבת העבודה מאפס.

3. **VS Code.** File → Open Folder → `module4_class` (מאשרים אמון אם נשאלים). פותחים את `class/show/three_artifacts.md` ואת `class/prompts.md` מה-Explorer; הם נשארים פתוחים ככרטיסיות. פותחים את Copilot Chat במצב Agent ובוחרים מודל. פותחים את הטרמינל (Ctrl+`) ומריצים `python -m pytest -q` : הפלט `16 passed` מאשר שהוא משתמש בסביבה הכיתתית. סביבת העבודה מגדירה גופנים בגודל 16 נקודות למקרן, שומרת קבצים אוטומטית, ומונעת מ-Copilot לשלוח את הקובץ הפתוח (בדרך כלל `prompts.md` ) יחד עם הפרומפטים; אם בתיבת הצ'אט עדיין מופיע תג של הקובץ הנוכחי, לוחצים עליו כדי לבטל אותו.
4. **הדפדפן.** כרטיסייה אחת עם כתובת ה- `wireframes.html` שה-setup הדפיס. בבלוק 1 פותחים את האפליקציה בכרטיסייה שנייה.
5. **המצגת** במצב הצגה רגיל, ו**התמליל** בטלפון.
6. **מומלץ: חזרה גנרלית אחת,** מתחילה ועד סוף, לפי `prompts.md` . אחריה, `python class/go.py start` מאפס את סביבת העבודה, כולל משימות שהוספת באפליקציה. כדי לבנות אותה מאפס במקום זאת, סוגרים קודם את VS Code ומריצים שוב את פקודת ה-setup.

**על המסך בשיעור:** המצגת ו-VS Code. כל פקודה רצה בטרמינל של VS Code, וכל פרומפט נשלח ל-Copilot Chat באותו חלון.

**התחנות.** כל `go` מבטל קודם שינויים שלא הושלמו בקוד, ולכן עוצרים את האפליקציה (Ctrl+C) ולוחצים Keep על עריכות צ'אט ממתינות לפני שמריצים אותו. הוא אף פעם לא נוגע ב- `class/` או בהגדרות של VS Code.

| `python class/go.py …` | מתי | מה נטען |
|---|---|---|
| `contract` | שלב 3, שקף 5 | החוזה המוסכם ב- `docs/` , וכללי העוזר |
| `clearing` | שלב 10, שקף 10 | הפיצ'ר, עם באג הניקוי המושתל |
| `0.4.0` | שלב 18, בלוק 5 | הגרסה ששוחררה, עם חבילת הבדיקות המלאה שלה |
| `0.4.1` | שלב 23, בלוק 7 | הגרסה עם ה"סידור" של ה-AI |
| `0.4.2` | שלב 31, בלוק 8 | הגרסה המתוקנת; שומרת גם את ה-patch שלה בשם `inputs/fix_0_4_2.diff` |
| `built` | חלופה, שלב 9 | הפיצ'ר המוכן, כשינויים שלא נעשה להם commit, לסקירה |
| `clock-test` | חלופה, שלב 26 | בדיקת השעון המוכנה, כתובה אך לא מורצת |
| `start` | אחרי חזרה גנרלית | נקודת הבסיס, כמו לפני השיעור |

## בלוק 1 · פתיחה (שקפים 3–1, 7 דק')

**שקף 1 (4 דק', יחד עם שלב 1).** מתחילים ב-VS Code על `class/show/three_artifacts.md`: שלוש שורות בודדות מאותו פרויקט. שואלים איזו מהן נכונה; מקבלים שתי תשובות. הנקודה: אף אחת מהן איננה נכונה או שגויה עד שמישהו מחליט, ואז ההחלטה צריכה להגיע לשלושתן.

**שלב 1 (30 שניות).** מציגים את המוצר, ה-Task Tracker ממודולים 2 ו-3, עדיין בלי תאריכי יעד. **פרומפט 1** מריצים את האפליקציה, פותחים בדפדפן את http://127.0.0.1:8000/, ועוצרים אותה עם Ctrl+C.

**שקפים 2 ו-3 (3 דק').** כמו בתמליל. בשקף 3 מבקשים מהתלמידים לשים לב להצעת AI אחת שתידחה או תתוקן היום; בסוף שואלים אותם עליה.

## בלוק 2 · דרישות (שקפים 4–5, 12 דק')

**שקף 4 (2 דק').** ממסגרים את הבקשה, ואז שואלים את הכיתה: "כמה החלטות חסרות העוזר ימצא? יותר או פחות מהכיתה הזו?"

**שלב 2, חי (4 דק').** פרומפט 2 העוזר קורא את בקשת בעל העניין ואת ה-backlog מתוך `inputs/`. קוראים את התשובה בקול. מסמנים החלטה חסרה אחת שהוא מצא ופריט backlog אחד שהוא תייג היטב.

חלופה: עוברים ישר ל"מה להדגיש" ומציינים את ההחלטות החסרות בעצמנו. "לפני הזמן הנוכחי" ("Before the current time") נשמע שלם, אבל לא אומר אם משימה שתאריך היעד שלה הוא היום נחשבת, או איזה שעון ואיזה אזור זמן חלים.

**מה להדגיש (3 דק').**

- סיפור משתמש: כחבר/ת צוות, אני רוצה תאריכי יעד אופציונליים ותצוגת Overdue only, כדי שאוכל לראות עבודה שלא הסתיימה ודורשת תשומת לב.
- החלטות פתוחות: יום קלנדרי או שעה מדויקת; איזה אזור זמן מגדיר את "היום"; האם משימה שתאריך היעד שלה הוא היום נחשבת באיחור; האם השמטת התאריך מעדכון שונה מניקוי שלו; איך יישלחו תזכורות.
- Backlog:

| # | פריט | תיוג | למה |
|---|---|---|---|
| 1, 10 | לשים דדליין על משימה / להוסיף תאריך ל-API | לשמור (keep, למזג) | אותו פריט |
| 2 | להפוך את תאריך היעד לחובה | לפסול (reject) | שובר כל משימה קיימת |
| 3 | להציג עבודה באיחור באדום | לפצל (split) | נדרשת תווית טקסט; הצבע רק תומך בה |
| 4 | להוסיף תצוגת Overdue only | לשמור (keep) | ליבת הבקשה |
| 5 | תזכורת במייל בכל בוקר | לדחות להמשך (defer) | דורש בירור: ערוץ, תזמון, הסכמה, בעלות |
| 6 | לאפשר למשתמשים להסיר דדליין | לשמור (keep) | ניקוי התאריך |
| 7 | לשמור על תפקוד המשימות הישנות | לשמור (keep) | מיגרציה תוספתית |
| 8 | להציג משימות שהושלמו כמושלמות | לשמור (keep) | משימה שהושלמה אף פעם אינה באיחור |
| 9 | להפוך דדליינים לקלים ואינטואיטיביים | לפצל (split) | לא ניתן לבדיקה בניסוחו; מפצלים לפריטים קונקרטיים, כמו כפתור Clear date גלוי ותוויות טקסט |

- דוגמת Given/When/Then: בהינתן משימה לא גמורה שתאריך היעד שלה 2026-10-04 ויום ייחוס 2026-10-04, כאשר קוראים אותה, אז `overdue` הוא `false`.

**שקף 5 (3 דק'). שלב 3.** עד עכשיו לא היה חוזה בסביבת העבודה, ולכן שלב הדרישות לא יכול היה לראות אותו.

**פרומפט 3** `go contract` יוצר את `docs/feature_contract.md`; פותחים אותו ועוברים על ההחלטות: תאריך קלנדרי, היום לפי UTC, היום אינו באיחור, משימה שהושלמה אף פעם אינה באיחור, השמטה לעומת null מפורש. משווים ל-Given/When/Then של ה-AI: באיזה כלל הוא נוגע, ומה שונה? מסיימים בהחלטת ה-UTC: פישוט מכוון שיחזור בסוף.

## בלוק 3 · תכן (שקפים 8–6, 12 דק')

שתי טיוטות התכן נעשות בשידור חי. כל פרומפט קובע מגבלת גודל, כי בלעדיה העוזר נוטה לצייר כל קובץ וכל הסתעפות, ואף אחד לא יכול לסקור את זה בכמה דקות. סוקרים כל טיוטה בעזרת רשימה קצרה וקבועה של שאלות, לא שורה אחר שורה. אם הצ'אט מציג טקסט Mermaid במקום ציור, קוראים את הטקסט; בגודל הזה הוא קצר.

**שקף 6 (2 דק').** שואלים: "האם העוזר ימציא לפחות רכיב אחד שלא קיים?" רוב האנשים עונים שלא.

**שלב 4, חי (4 דק').** **פרומפט 4** בודקים את הדיאגרמה רק בעזרת ארבע השאלות שבשקף: אילו רכיבים משתנים, מה חדש, מאיפה מגיע "היום", ודרך מה עובר תאריך שנוקה. גם שאלה שהדיאגרמה לא יכולה לענות עליה היא ממצא. מה לחפש: scheduler או שירות התראות מומצאים, עמודת `overdue` שמורה, כלל שקורא את השעון של הדפדפן או שאין לו שעון בכלל, חץ שמצביע בכיוון ההפוך.

(אפשר לקצר.) אם הדיאגרמה יוצאת גדולה בכל זאת: בהרצת ניסיון בלי מגבלת הגודל צוירו 15 תיבות ברמת קובץ. ארבע השאלות עדיין מצאו שתי בעיות בתוך דקה. כלל ה-overdue קיבל `now`, ואף תיבה לא אמרה מאיפה `now` מגיע; וה-repository ביצע את סינון ה-overdue, מה שמזמין שעון שני בתוך ה-SQL. דיאגרמה גדולה היא דרך טובה להמחיש למה צריך את המגבלה.

חלופה: עוברים לשקף 7 ומציינים את הטעויות הנפוצות מהרשימה שלמעלה.

**שקף 7 (2 דק').** תכן הייחוס: השינוי נכנס בתוך החלקים הקיימים. הדפדפן שולח תאריך או null, ה-API מאמת אותו, ה-repository שומר רק את התאריך, ו-"overdue" מחושב בכל קריאה מפונקציית שעון אחת שמחזירה את היום הנוכחי לפי UTC. דגל שמור היה מתיישן כשהיום מתחלף או כשמשימה מושלמת. אחר כך עוקבים אחרי מסלול הניקוי לאורך השורה התחתונה: הדפדפן שולח null, ה-API צריך לשמור אותו ולא להשמיט אותו, וה-repository שומר את התאריך המנוקה. ההוכחה איננה התגובה לעדכון; היא קריאה מאוחרת יותר שמראה שהתאריך נעלם. הקריאה המאוחרת הזו היא כל הטעם של כלל הניקוי.

**שקף 8 ושלב 5, חי (4 דק').** Wireframe אמור לעזור להחליט איך ישתמשו בפיצ'ר; טבלת מצבים לא מראה איך מישהו מזין תאריך או מנקה אותו. השקף מציג את הסידור שבו משתמש הלקוח הרץ. כפתור Clear date המפורש חשוב: ריקון שדה תאריך מובנה של הדפדפן עובד אחרת בכל דפדפן, ומשתמשים רבים אף פעם לא מגלים איך עושים את זה.

**פרומפט 5** העוזר כותב מחדש את `wireframes.html`; מרעננים את כרטיסיית הדפדפן. בודקים את שתי הפריסות עם הכיתה:

1. אפשר לקבוע תאריך בעת הוספת משימה.
2. Clear date הוא פקד גלוי במשימה שיש לה תאריך, ולא מוסתר בתפריט.
3. יש מסנן Overdue only.
4. הסטטוס מוצג בטקסט ("Overdue, Done, "Due) ואחריו התאריך, No due date), ולא רק בצבע.
5. אין שום דבר מחוץ לתחום: אין תזכורות, אין שעות ביום.

טיוטות AI נכשלות בדרך כלל באחד מאלה, לרוב כשהן מסמנות overdue באדום בלבד או מחביאות את הניקוי. (אפשר לקצר:) ואז משווים: A שם את כל הפקדים בשורה (קל לסריקה, עמוס ברשימה ארוכה; זו הפריסה של הלקוח הרץ); B יכול להפוך את Overdue ללשונית עם מונה, אבל מסתיר את התאריך עד שבוחרים משימה, וייתכן שבהוספת משימה כבר לא אפשר יהיה לקבוע תאריך בצעד אחד.

אם העוזר מדפיס את ה-HTML בצ'אט במקום לכתוב את הקובץ, אומרים לו: "Write it to wireframes.html." חלופה: שקף 8 מציג את פריסה A; מתארים את B במשפט אחד ומריצים את רשימת הבדיקה על A.

## בלוק 4 · בנייה ובדיקות (שקפים 9–11, 26 דק')

**שקף 9 · הבנייה (11 דק'), שלבים 6–9.** החוזה כבר נמצא בסביבת העבודה.

- **שלב 6:** `פרומפט 6` קודם `go contract`, כדי שהבנייה תתחיל מתחנה נקייה ודף ה-wireframe לא יופיע בסקירה. אחר כך העוזר מתכנן ומחכה. קוראים את התוכנית בקול ומחפשים קובץ שלא אמור להשתנות, תלות חדשה, או כל דבר מחוץ לחוזה, כמו תזכורות.
- **שלב 7:** `פרומפט 7` מצמצמים אם צריך, ומאשרים. בזמן שהוא עורך, מסבירים את הסדר: תכנון, אישור, עריכה, סקירה מול החוזה, ואז בדיקה. העריכות לוקחות כמה דקות; **אם הן לא הסתיימו כ-5 דקות אחרי האישור (בסביבות 0:38), עוברים לחלופה.**
- כשהוא מסיים, **לא פותחים את קוד ה-PATCH שב-`app/repository.py`.** זו ההדגמה הבאה.
- (אפשר לקצר.) **שלב 8:** `פרומפט 8` מבקשים מהעוזר סקירה ראשונה לפני שמציגים את השינויים. בין אם מריצים את השלב ובין אם לא, מעבירים את הנקודה: כך צוותים רבים משתמשים היום ב-AI ב-pull requests – כקורא ראשון ומהיר, לא כמאשר.
- מציגים את השינויים בתצוגת Source Control של VS Code, שכוללת גם קבצים חדשים כמו `app/overdue.py`. עוברים על ארבע השאלות שבשקף: מיגרציה תוספתית, frontend בתוך המודולים הקיימים, overdue רק כשהתאריך מוקדם ממש מיום הייחוס ואף פעם לא למשימות שהושלמו, שעון UTC אחד.
- **שלב 9:** `פרומפט 9` מריצים את האפליקציה, מוסיפים תאריך, מנקים אותו, עוצרים את השרת. **אם הניקוי לא נשמר, לא מתקנים:** "זו בדיוק ההדגמה הבאה."

חלופה: אומרים שעוברים לתוצאה המוכנה. `פרומפט 9, שורה אחרונה` `python class/go.py built` מבטל את השינויים החיים שלא הסתיימו ומניח את הפיצ'ר המוכן כשינויים שלא נעשה להם commit, כך שסוקרים אותו ב-Source Control בדיוק כאילו נבנה בשידור חי.

**שקף 10 · קודם בדיקה, אחר כך תיקון (10 דק'), שלבים 15–10.**

- **שלב 10:** `פרומפט 10` `go clearing` טוען את הגרסה המוכנה של הפיצ'ר עם פגם מושתל אחד, מהסוג שסקירה יכולה לפספס. מסבירים: PATCH ששולח null מפורש לא מנקה את התאריך השמור, ובכל זאת 16 הבדיקות המקוריות עוברות, כי הן נכתבו לפני שהיו תאריכי יעד. שואלים: "שתיים מתוך 16 הבדיקות שולחות עדכונים. האם אחת מהן תתפוס את זה?" (לא: אף אחת מהן לא שולחת תאריך יעד.)
- **שלב 11: את הבדיקה כותב המנחה, לא העוזר.** אם מבקשים מהעוזר קודם, הוא מוצא ומתקן את הבאג בפחות מדקה, ואף אחד לא לומד שום דבר על הדרישה. `פרומפט 11` מדביקים את הבדיקה ב-`tests/test_student_feature.py` ועוברים עליה שורה אחר שורה, תוך שאלת הכיתה מה כל assertion צריך להיות.
- **שלב 12:** `פרומפט 12` צפוי `1 failed`, עם `assert '2026-10-03' is None`: היא נכשלת מהסיבה שציפינו לה. מצביעים על הקריאה שאחרי העדכון: בדיקה שהייתה בודקת רק את תגובת ה-PATCH (200) הייתה עוברת ולעולם לא תופסת את זה.
- **שלבים 13 ו-14:** `פרומפט 13` העוזר מסביר את הכישלון ומציע את התיקון הקטן ביותר; קוראים את ההצעה, ואז `פרומפט 14` מאשרים. ייתכן שהוא יריץ את חבילת הבדיקות בעצמו אחר כך.
- **שלב 15:** `פרומפט 15` `git diff app/repository.py` מציג את השורה האחת שהשתנתה. ציון שם הקובץ משאיר את הבדיקה שכתבת, שעוד לא נעשה לה commit, מחוץ ל-diff. חבילת הבדיקות המלאה: `17 passed`.

חלופה: `פרומפט 14, שורה אחרונה` `git restore --source 0.4.0 app/repository.py` מכניס את השורה המתוקנת. אחר כך סוקרים אותה כרגיל, `פרומפט 15`. השינוי:

```
-        update_data = payload.model_dump(exclude_unset=True, exclude_none=True)
+        update_data = payload.model_dump(exclude_unset=True)
```

**שקף 11 · תוכנית בדיקות, נתוני בדיקה ובדיקת הבדיקות (5 דק'), שלבים 17–16.**

1. התיקון (1 דק'). `exclude_none` זורק את ה-null המפורש, ולכן התאריך הישן שורד. `exclude_unset` שומר null שנשלח, ומשמיט רק שדות שמעולם לא נשלחו.
2. שלב 16, תוכנית בדיקות ונתוני בדיקה בשידור חי (3 דק'). `פרומפט 16` מסירים יחד עם הכיתה כפילות אחת, ובודקים תוצאה צפויה אחת מול החוזה ולא מול הקוד. חלופה: הטבלה שבשקף.
3. שלב 17, בדיקה חלשה אחת (1 דק'). `פרומפט 17` את `class/show/weak_test_example.py` כתב AI, והיא עוברת. אבל היא בודקת רק ספירה, כך שגם משימה בודדת שגויה הייתה עוברת. בדיקה טובה מציינת את המשימה המדויקת שהיא מצפה לה, מקלטים קבועים, כשהתוצאה הצפויה לקוחה מהחוזה. ואז: "איך נדע שהבדיקות שלנו תופסות כלל שגוי? נבדוק את הבדיקות בבלוק 7."

## בלוק 5 · שחרור (שקפים 13–12, 10 דק')

**שלב 18:** פרומפט 18 `go 0.4.0` טוען את הגרסה ששוחררה, עם חבילת הבדיקות המלאה שלה. כל מה שבבלוק הזה רץ עליה.

**שקף 12 (3 דק').** ל-"AI ב-pipeline" יש שתי משמעויות: עוזר שכותב וסוקר את ה-workflow (היום), וסוכן שרץ בתוכו (מודול 6). בשקף מוצגת טיוטת ה-workflow של ה-AI. שואלים: "מה ה-workflow הזה לא בודק?" נותנים לזה 30 שניות. הוא מריץ רק Python, בעוד שהשינוי נגע בארבעה מודולי JavaScript, בדף ובגיליון הסגנונות, והוסיף בדיקות frontend; ה-workflow המתוקן מוסיף job בשם `frontend` . מסיימים בשלושת סוגי הראיות: קובץ workflow אומר מה מתוכנן, הרצה מקומית מראה מה רץ כאן, ולוג מתארח (hosted) מראה מה רץ עבור commit אחד.

**שקף 13 (7 דק').** בדיקת ה-Health אומרת שהתהליך עונה; ה-probe העסקי בודק כלל מוצר אחד מול השרת הרץ. שואלים: "תנו דוגמה לגרסה שגויה שה-probe הזה היה מעביר." מקבלים שתי תשובות. **שלב 19:** פרומפט 19 העוזר נותן דוגמה אחת, בארבע שורות לכל היותר.

**מה להדגיש** (וזו גם החלופה אם התשובה איטית). ה-probe משתמש במשימה שהושלמה שתאריכה עבר, ומשימה כזו אף פעם אינה באיחור, בשום יום. לכן הוא מעביר גרסה שטועה ב"היום" סביב חצות, שמתייחסת להיום כאיחור, ששוברת את הניקוי, או ששוברת את ה-frontend. שומרים את הרשימה הזו: הבלוק הבא חוזר אליה.

**שלב 20,** החזרה הגנרלית לשחרור. פרומפט 20 צפוי: `34 passed` , ואחריו `Local rehearsal passed: full suite, health, completed-task probe, cleanup.` (אפשר לקצר:) rollback פירושו לשחזר את הקוד הקודם ולשמור את הנתונים; העמודה החדשה יכולה להישאר, כי קוד ישן יותר מתעלם ממנה.

## בלוק 6 · תחזוקה: מיון התקלה (שקפים 15–14, 12 דק')

**שקף 14 (2 דק').** הגרסה יצאה, ומגיע שרשור תמיכה מהצוות בתל אביב. פותחים את `inputs/support_thread.md` ל-30 שניות: עשר הודעות קצרות, שתיים מהן רעש.

**שלב 21, חי (4 דק').** הפרומפט מורה לעוזר לקרוא רק את השרשור, כך שהוא עונה מתוך הראיות ולא מתוך הקוד. **פרומפט 21** בודקים שהתשובה שמרה על המשימה המדויקת, תאריך היעד והשעה: תשובה שאומרת "כמה משתמשים דיווחו על בעיה בלילה" איבדה את כל מה שחשוב.

חלופה: העובדות שלהלן, כשקוראים אותן מהשרשור שעל המסך.

**מה להדגיש (2 דק').** שואלים קודם: "האם זו אחת הגרסאות השגויות מרשימת ה-probe?"

- עובדות: ב-21:30 UTC, שהן 00:30 בתל אביב, `task-42` (תאריך יעד 3 באוקטובר, לא הושלמה) מוצגת כ-Overdue; התאריך השמור לא השתנה; משימה חדשה, שמעולם לא נערכה, מתנהגת אותו דבר; ה-API עצמו מחזיר `overdue: true` ; השרת מוגדר לשעון מקומי, והוא והמחשבים הניידים של הצוות מראים 00:30 ב-4 באוקטובר; השינוי היחיד היה 0.4.1, "סידור" בסיוע AI; מ-03:00 שעון מקומי, שהיא חצות UTC, הכול נראה כמצופה.
- השערות: "היום" נלקח מהיום המקומי של השרת (הסבירה ביותר); העריכה שינתה משהו (נשללת בגלל המשימה שמעולם לא נערכה); הדפדפן מחליט בעצמו על overdue, לפי השעון שלו או לפי ערך שמור במטמון (התזמון לא יכול לשלול את זה, כי המחשבים הניידים מראים את אותה שעה כמו השרת; התשובה של ה-API עצמו שוללת).
- רעש: חיפוש איטי בלשונית Done, ובקשת תזכורת המייל.
- בדיקה מבחינה: קוראים לפונקציה האמיתית שמחליטה מהו "היום" ברגע קבוע מעט אחרי חצות מקומית (00:30 ב-UTC+03:00) ומשווים את התשובה שלה ליום לפי UTC, 3 באוקטובר. אין צורך לחכות לחצות הבאה.

**שקף 15 (4 דק'), שלב 22.** אותו רגע נופל על שני ימים: ב-21:30 UTC ב-3 באוקטובר, שעון שרת שמקדים בשלוש שעות מראה 00:30 ב-4 באוקטובר. **פרומפט 22** ה-diff מראה מה 0.4.1 שינתה. מלבד מספר הגרסה, זו שורת לוגיקה אחת: ההמרה ל-UTC הוסרה, יחד עם ה-import שהיא נזקקה לו, וה-docstring עדיין אומר UTC. שואלים: "האם סוקר אנושי היה תופס את ה-pull request הזה?" כל הבדיקות עברו כי ה-fixture `client` של הבדיקות מחליף את `get_reference_date` ב-4 באוקטובר קבוע, וה-probe משתמש במשימה שהושלמה. כל בדיקה צדקה לגבי מה שהיא כיסתה; אף אחד לא בדק את החלק שהבדיקות החליפו.

אם מישהו אומר ששרתים רצים ב-UTC: רובם כן, וזה בדיוק מה שהופך את הבאג הזה למסוכן. במכונה שרצה ב-UTC, כמו CI runner טיפוסי או שרת ענן, ההמרה שהוסרה לא משנה כלום, ולכן "no behavior change intended" נראה נכון. 0.4.0 לא הייתה תלויה באזור הזמן של המארח; 0.4.1 תלויה בו, והיא נכשלת רק בשרת שאינו על UTC, כמו השרת של המשרד הזה.

## בלוק 7 · סגירת המעגל (שקף 16, 12 דק')

**שלב 23 (1 דק'):** **פרומפט 23** `go 0.4.1` . מסבירים את המטרה: להחליף רק את השעון ב-00:30 ב-4 באוקטובר ב-UTC+03:00, לקרוא לפונקציית "היום" האמיתית, ולצפות ל-3 באוקטובר.

**שלבים 24–26, חי (4 דק').**

- **שלב 24:** **פרומפט 24** העוזר מציע בדיקה ומחכה. קוראים אותה עם הכיתה: רק `server_now` מוחלף, לזמן יש אזור זמן, הפונקציה האמיתית `get_reference_date()` נקראת, והיום הצפוי לקוח מהחוזה. שואלים: "האם היא תעבור על 0.4.1?"
- **שלב 25:** **פרומפט 25** מאשרים. העוזר כותב את הקובץ ועוצר.
- **שלב 26:** **פרומפט 26** מריצים אותה. צפוי: היא נכשלת עם `assert datetime.date(2026, 10, 4) == datetime.date(2026, 10, 3)` . **אם היא עוברת במקום זאת,** העוזר השתמש ב-fixture `client` או החליף את `get_reference_date` . זה רגע הלימוד הטוב ביותר האפשרי: מראים איזה fixture החליף מה.

חלופה: **פרומפט 26, שורה אחרונה** `python class/go.py clock-test` כותב את הבדיקה המוכנה לקובץ. אחר כך מריצים את הבדיקה עם השורה הראשונה של **פרומפט 26** .

**שלבים 27–29, התיקון (2 דק').** **פרומפט 27** העוזר מסביר את התוצאה ומציע את התיקון הקטן ביותר; **פרומפט 28** מאשרים אותו. **פרומפט 29** `git diff app/main.py` מראה את שתי השורות ששוחזרו, וחבילת הבדיקות המלאה נותנת `35 passed` .

חלופה: **פרומפט 28, שורה אחרונה** `git restore --source 0.4.2 app/main.py` מכניס את הקובץ המתוקן. אחר כך סוקרים אותו כרגיל, **פרומפט 29** ; ה-diff שלו מראה גם את שינוי מספר הגרסה. השינוי:

```
-from datetime import date, datetime
+from datetime import date, datetime, timezone
 ...
-    return server_now().date()
+    return server_now().astimezone(timezone.utc).date()
```

**שלב 30, בדיקת המוטציה (3 דק').** עכשיו בודקים את הבדיקות. Mutation testing פירושו לשנות את הקוד בכוונה, בצורה קטנה ושגויה, ולוודא שבדיקה כלשהי שמה לב. המוטנט שלנו הופך "קטן מ-" ל"קטן או שווה ל-" בכלל ה-overdue, כך שמשימה שתאריך היעד שלה הוא היום נחשבת באיחור. שואלים: "אילו בדיקות נכשלות? האם בדיקת השעון החדשה שלנו נכשלת?" **פרומפט 30** מריצים. צפוי: `Detected` , כשבדיקות גבול-היום, הסינון וזמן-הקריאה נכשלות ( `3 failed, 32 passed` ). בדיקת השעון עוברת, כי היא בודקת איזה יום עכשיו, ולא את כלל ה-overdue. (הכלי בודק קודם שחבילת הבדיקות ירוקה; אם תיקון השעון לא נכנס, הוא אומר זאת במקום להריץ את המוטנט.)

**למה 34 בדיקות שעוברות פספסו את זה (2 דק'), השאלה שבשקף.** כל בדיקת API רצה דרך ה-fixture `client` , שמחליף את `get_reference_date` . ההחלפה הפכה את הבדיקות לצפויות, וגם הוציאה את הפונקציה ממה שהן בדקו. טעויות נפוצות בכתיבת בדיקת השעון: שימוש ב-fixture `client` (הבדיקה לעולם לא יכולה להיכשל), patch ל- `get_reference_date` עצמה, datetime נאיבי בלי `tzinfo` , וקריאה ל- `datetime.now()` בתוך הבדיקה.

## בלוק 8 · תיקון ומסירות (שקף 17, 8 דק')

**שלב 31 (1 דק'):** **פרומפט 31** `go 0.4.2` והחזרה הגנרלית: `37 passed`. פקודת ה-`go` שומרת גם את ה-patch האמיתי של 0.4.2 → 0.4.1 בשם `inputs/fix_0_4_2.diff`.

**שלב 32, חי (2 דק').** **פרומפט 32** העוזר קורא את ה-patch האמיתי ואת הבדיקות החדשות, לא סיכום, ומנסח שלושה טקסטים לשלושה קהלים.

**סקירה (3 דק').** קוראים כל משפט ושואלים אם ה-patch או הבדיקות תומכים בו; מוחקים כל מה שהם לא תומכים בו, כמו "improved performance". תשובות טובות נראות כך, והן גם החלופה אם התשובה איטית:

- יומן שינויים פנימי (0.4.2): תוקן יום הייחוס של overdue. גרסה 0.4.1 השתמשה בתאריך המקומי של השרת במקום בתאריך לפי UTC. נוספו בדיקות רגרסיה שמחליפות רק את השעון וקוראות לפונקציית יום-הייחוס האמיתית.
- הערת שחרור למשתמשים: תוויות Overdue מתחלפות שוב בשעה המוסכמת. בגרסה 0.4.1, חלק מהמשימות הוצגו כבאיחור כמה שעות מוקדם מדי, זמן קצר אחרי חצות מקומית.
- הערה ל-README: למשימות יכול להיות תאריך יעד אופציונלי (`YYYY-MM-DD`). שולחים `"due_date": null` כדי לנקות אותו; השמטת השדה משאירה אותו. משימה היא באיחור כשהיא לא הושלמה והתאריך שלה מוקדם מהיום הנוכחי לפי UTC. לדוגמה, ביום 2026-10-04 UTC, משימה שתאריך היעד שלה 2026-10-03 נמצאת באיחור, ומשימה שתאריך היעד שלה 2026-10-04 אינה באיחור.

**ההחלטה הבאה (2 דק').** מצביעים על הפריט האחרון בשקף 17, שאלת ה-backlog: האם "היום" צריך לעקוב אחרי אזור הזמן של הצוות? הצוות בתל אביב עובד לפי ימים מקומיים, כך שבמשך שלוש שעות בכל לילה המוצר התקין לא מסכים עם שעון הקיר שלהם. האם החוזה צריך להשתנות – זו החלטת מוצר, ולכן היא חוזרת ל-backlog ולא נכנסת ל-hotfix. אם הצוות בוחר בשעון מקומי, שומרים אזור זמן בעל שם כמו `Asia/Jerusalem`, ולא היסט קבוע, כי ההיסט משתנה עם שעון הקיץ.

## בלוק 9 · סיכום (שקפים 18–20, 10 דק')

**שקף 18 (2 דק')** עוקב אחרי כלל הניקוי דרך כל התוצרים: הדוגמה שבחוזה, ה-null שעובר דרך התכן, הקוד ששומר null שנשלח, הבדיקה שקוראת אחרי העדכון, והערת השחרור שמשתמשת באותה דוגמה.

**שקף 19 (3 דק').** שואלים: "איזו הצעת AI ראיתם היום שנדחתה או תוקנה, והאם גם אתם הייתם נוהגים כך?" ואז מודולים 5, 6 ו-8, כמו בתמליל.

**שקף 20 (5 דק').** שאלת היציאה כדיון כיתתי: בוחרים דרישה אחת; מציינים תוצאה שגויה סבירה, בדיקה שהייתה תופסת אותה, ובדיקה שהייתה מפספסת אותה. אם הכיתה שקטה, מציעים:

- **ניקוי:** עדכון עם null מחזיר 200 אבל התאריך שורד. נתפס על ידי בדיקה שמנקה וקוראת שוב; מפוספס על ידי 16 הבדיקות המקוריות ועל ידי ה-probe, שאף פעם לא מנקה דבר.
- **היום:** מעט אחרי חצות מקומית, משימה שתאריך היעד שלה אתמול לפי UTC מוצגת כבאיחור. נתפס על ידי בדיקה שמחליפה רק את השעון; מפוספס על ידי כל בדיקה שמשתמשת ב-fixture `client`, ועל ידי ה-probe של המשימה שהושלמה.

מסיימים באמירה שה-zip של התלמידים וקבצי המשימות ישותפו אחרי השיעור, לכל מי שרוצה לבצע את שש המשימות בעצמו.

## נספח · הבדלים מהמודול בן ארבע השעות

| מודול ארבע השעות | גרסת השעתיים |
|---|---|
| שש משימות בזוגות (A–F) והתחקירים שלהן | כל שלב מורץ בשידור חי; הכיתה מנבאת קודם; נקודות התחקיר הופכות ל"מה להדגיש" |
| התלמידים מכינים סביבה, פורסים שתי תיקיות משימה ועושים להן commit | התלמידים לא צריכים כלום; ה-zip של התלמידים וארבעת קבצי המשימות משותפים אחרי השיעור |
| תיקיית הדגמה ועוד קבצים מכמה תיקיות בחבילה | סביבת עבודה כיתתית אחת: כל שלב ב-`class/prompts.md`, כל תחנה בפקודת `go` אחת |
| שרשור תמיכה, יומן בקשות ורשומת שחרור (כ-35 שורות) | שרשור קצר אחד (עשר הודעות) ורגע אחד: 00:30 בתל אביב הוא 21:30 UTC |
| דיאגרמת ארכיטקטורה, דיאגרמת רצף ו-wireframes | דיאגרמת הארכיטקטורה וה-wireframes; מסלול הניקוי מוסבר בשקף 7 |
| שתי בדיקות חלשות; המוטנט בבלוקים 4 ו-7 | בדיקה חלשה אחת; המוטנט פעם אחת, בבלוק 7 |
| טיוטת ה-CI וה-workflow המתוקן, פתוחים זה לצד זה | הטיוטה בשקף 12; הפער במשפט אחד |
| שתי הפסקות ועתודה של 12 דקות | בלי הפסקה; עתודה של 11 דקות |

## נספח · תשובות לשיעורי הבית

שיעורי הבית נמצאים בסוף מדריך התלמיד.

1. **שש המשימות:** התשובות הן רשימות "מה להדגיש" שבמדריך הזה, בלוק אחר בלוק. משימה E משתמשת בראיות המלאות יותר שב-`student/inputs/maintenance/`; העובדות שלה מוסיפות שעות מדויקות לאלה שבבלוק 6.
2. **בירור לגבי תזכורות:** ערוץ והסכמה, לפי איזה אזור זמן נקבע "בוקר", ניסיונות חוזרים וכפילויות, מי אחראי על משלוחים שנכשלו, ועלות. להשאיר את התזכורות נפרדות מתאריכי היעד.
3. **אזור זמן עסקי:** לשנות את REQ-04 לאזור זמן בעל שם (`Asia/Jerusalem`), לשמור על שעון יחיד, ולבדוק את שני צדי חצות המקומית ואת מעבר השעון ב-25 באוקטובר 2026 (מ-UTC+03:00 ל-UTC+02:00). ב-Windows, `zoneinfo` צריך את החבילה `tzdata`.
4. **מסירה בין תפקידים:** המוצר מוסר דוגמאות מוסכמות, המפתחים מוסרים את השינוי והערת תכן, QA מוסר תוצאות צפויות בלתי תלויות, התפעול מוסר בדיקות ותוכנית rollback, והתמיכה מוסרת עובדות שנצפו – כולם עם אותם מזהי דרישה.

# נספח · פתרון תקלות

| בעיה | פתרון |
|---|---|
| תשובה לוקחת יותר מכ-60 שניות | להשתמש בחלופה של השלב ולומר שהיא הוכנה מראש |
| העוזר מבקש להריץ פקודה | לאשר; התוצאה מופיעה בטרמינל |
| העוזר עורך לפני האישור | לסקור את ה-diff כרגיל; ה-`go` הבא מבטל כל מה שחורג מהתסריט |
| העוזר כותב או משנה בדיקות כדי שיעברו | רגע לימוד טוב: לדחות (Undo) ולהצביע על הכלל ב-`AGENTS.md` |
| העוזר מדפיס את ה-HTML של ה-wireframe בצ'אט | לומר לו: "Write it to wireframes.html." |
| `go.py` נעצר עם שגיאת Git | לעצור את האפליקציה (Ctrl+C) ולהריץ שוב |
| תוכן ישן מופיע שוב בקובץ אחרי `go` | עריכת צ'אט ממתינה בוטלה מאוחר יותר. ללחוץ Keep לפני כל `go`; להריץ שוב את אותו `go` כדי לתקן |
| שלב 12 או 26 מדווח "no tests ran" | הקובץ לא נכתב או לא נשמר: לשמור אותו, או להשתמש בחלופה של השלב |
| קבצי `class/` לא מופיעים ב-Ctrl+P, ו-Copilot לא יכול לקרוא אותם | שניהם בכוונה: `class/` מכילה את `prompts.md`. פותחים את הקבצים שבה מה-Explorer |
| הרצה חוזרת של ה-setup נכשלת ב-Windows | לסגור את VS Code (הטרמינל שלו מחזיק את התיקייה פתוחה) ולהריץ שוב |
| פורט 8000 תפוס | להוסיף `-- port 8001`–ולפתוח את הפורט הזה במקום |
| VS Code לא מוצא את הסביבה | `setup_class.py` מגדיר אותה עבור סביבת העבודה; אחרת מריצים `python -c "import sys; print(sys.executable)"` ומדביקים את הנתיב ב-**Python: Select Interpreter → Enter interpreter path…** |